Theory of Computation

Lecture 8

Context-free Grammar

$$G = (N, \Sigma, P, S)$$

$$S \rightarrow aSb \mid \varepsilon$$

$$S \rightarrow [S] \mid SS \mid \varepsilon$$

$$S \rightarrow aSa \mid bSb \mid \varepsilon$$

Closure Properties

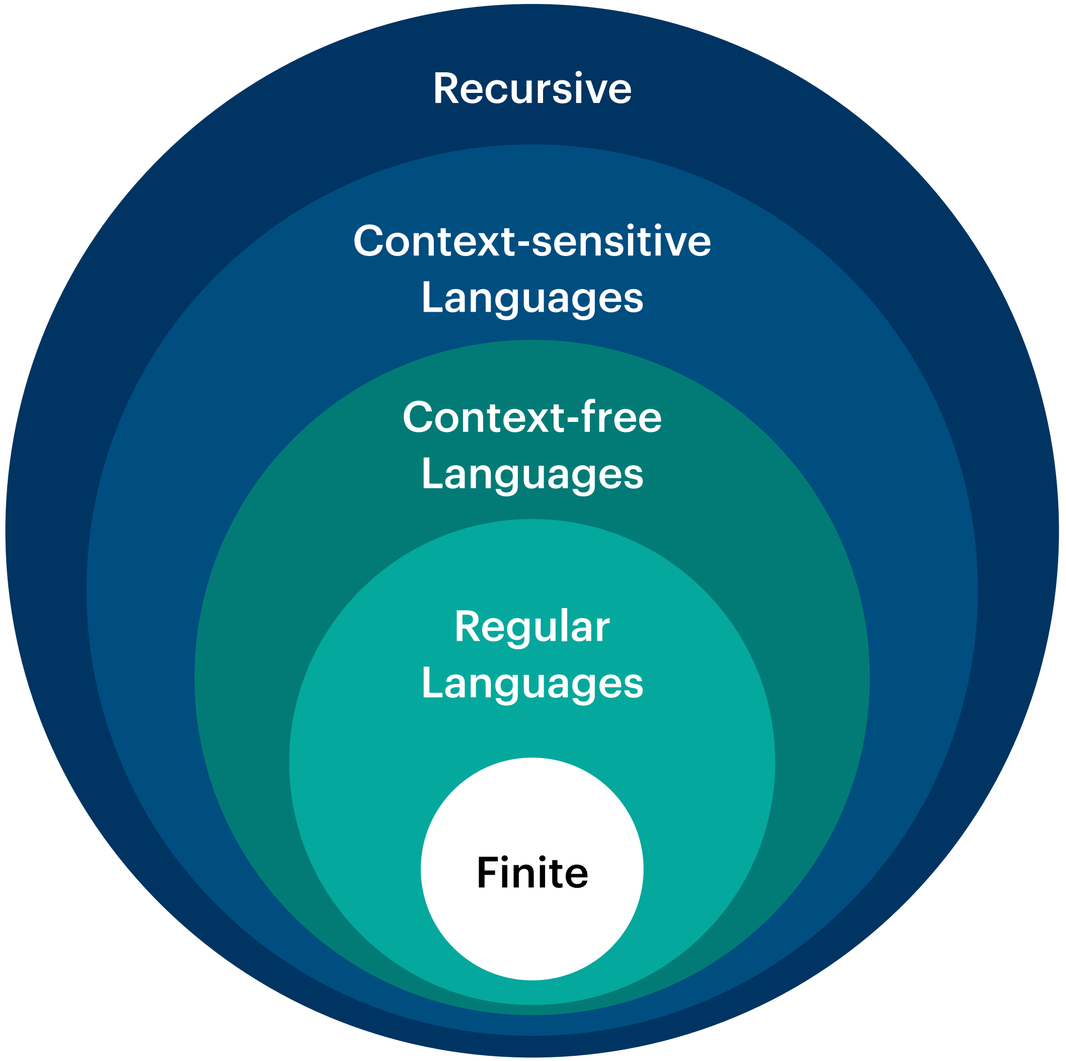
1. Union
2. Concatenation
3. Kleene star
4. Reversal
5. Homomorphism
6. Reverse homomorphism
7. Permutation
8. Intersection with Regular Languages

Not closed under:

1. Intersection
2. Complement

The Chomsky Heirarchy

Noam Chomsky



انظر إلى ما قال ولا
تنظر إلى من قال

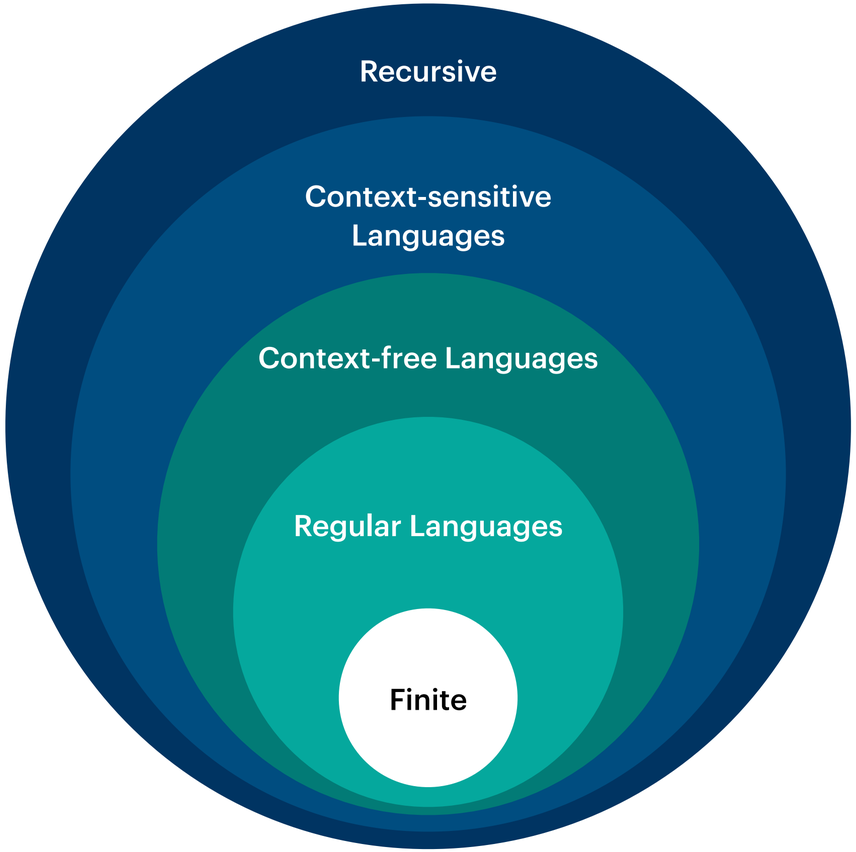
Noam Chomsky

Newly released files shed new light on Chomsky and Epstein relationship

Latest communications undermine Chomsky's earlier claims that he primarily had financial dealings with Epstein

Two Ontological Types

Noam Chomsky



Chomsky Hierarchy

Two Ontological Types

Type:

A human being, moral agent

Evaluation depends on:

Ethical standards

Social impact

Historical context

Noam Chomsky

Type:

A mathematical classification

Validity depends on:

Formal definitions and proofs

Internal consistency

True or false independent of the author

Chomsky Hierarchy

Exercises:

88. Given a context-free grammar $G = (N, \Sigma, P, S)$ and a DFA $A = (Q, \Sigma, \delta, s, F)$, define a grammar $G' = (V', \Sigma, P', S')$ such that $\mathscr{L}(G') = \mathscr{L}(G) \cap \mathscr{L}(A)$. Hint: Consider $V' \subseteq V \times Q \times Q$.

89. A context-free grammar is linear if each production rule leads to only one non-terminal. Develop a pigeonhole argument to prove that the language of well-balanced parentheses is not a linear CFL.

90. Show that $\{a^n b^m c^{n+m} : n, m \in \mathbb{N}\}$ is a context-free language.

91. Show that $\{a^p : p \text{ is prime}\}$ is not a context-free language.

Decision Problems:

Emptiness

Decision Problems:

Emptiness

1. Mark all terminals.
2. Repeatedly, if there is a rule $A \to \alpha$, with every character α is marked, then mark A.
3. Continue until no new nonterminals can be marked.
4. Check if the start symbol S is marked.

Decision Problems:

Emptiness

1. Mark all terminals.
2. Repeatedly, if there is a rule $A \to \alpha$, with every character α is marked, then mark A.
3. Continue until no new nonterminals can be marked.
4. Check if the start symbol S is marked.

$$\mathscr{L}(G) \neq \varnothing \iff S \text{ is marked.}$$

Decision Problems:

Emptiness

Universality?

1. Mark all terminals.
2. Repeatedly, if there is a rule $A \to \alpha$, with every character α is marked, then mark A.
3. Continue until no new nonterminals can be marked.
4. Check if the start symbol S is marked.

$$\mathscr{L}(G) \neq \varnothing \iff S \text{ is marked.}$$

Decision Problems:

Membership

$$S \to aAb \mid AS \mid ab$$

$$A \to S \mid Ab$$

Is $abbab \in \mathscr{L}(G)$?

Decision Problems:

Membership

Membership for CFLs is fundamentally a *parsing* problem.

Cocke–Younger–Kasami Algorithm

Earley Algorithm

$$O\left(|G| \times |w|^3\right)$$

Cocke–Younger–Kasami Algorithm

Step 1: Convert to Chomsky Normal Form (CNF)

Cocke–Younger–Kasami Algorithm

Step 1: Convert to Chomsky Normal Form (CNF)

$$A \to BC$$

$$A \to a$$

And $\varepsilon \notin \mathscr{L}(G)$.

Every context-free grammar can be written as a CNF

Cocke–Younger–Kasami Algorithm

Step 1: Convert to Chomsky Normal Form (CNF)

$$S \rightarrow aAb \mid AS \mid ab$$

$$A \rightarrow S \mid Ab$$

Cocke–Younger–Kasami Algorithm

Step 1: Convert to Chomsky Normal Form (CNF)

$$S \rightarrow V_a A V_b \mid AS \mid V_a V_b$$

$$A \rightarrow S \mid A V_b$$

$$V_a \rightarrow a$$

$$V_b \rightarrow b$$

Cocke–Younger–Kasami Algorithm

Step 1: Convert to Chomsky Normal Form (CNF)

$$S \rightarrow V_a A V_b \mid AS \mid V_a V_b$$

$$A \rightarrow S \mid Y$$

$$Y \rightarrow A V_b$$

$$V_a \rightarrow a$$

$$V_b \rightarrow b$$

Cocke–Younger–Kasami Algorithm

Step 1: Convert to Chomsky Normal Form (CNF)

$$S \rightarrow V_a Y \mid SS \mid YS \mid V_a V_b$$

$$Y \rightarrow SV_b \mid YV_b$$

$$V_a \rightarrow a$$

$$V_b \rightarrow b$$

Cocke–Younger–Kasami Algorithm

Step 1: Convert to Chomsky Normal Form (CNF)

Insight: Some variable must derive letter of w

Cocke–Younger–Kasami Algorithm

Step 1: Convert to Chomsky Normal Form (CNF)

Step 2: Build a DP table.

Cocke–Younger–Kasami Algorithm

Step 1: Convert to Chomsky Normal Form (CNF)

Step 2: Build a DP table.

i = starting position

k = substring length

$T[i, k]$ = set of variables that derive substring

$$w_i \cdots w_{i+k-1}$$

Cocke–Younger–Kasami Algorithm

$$S \rightarrow V_a Y \mid SS \mid YS \mid V_a V_b$$

$$Y \rightarrow S V_b \mid Y V_b$$

$$V_a \rightarrow a$$

$$V_b \rightarrow b$$

	1	2	3	4	5
a					
b					
b					
a					
b					

Cocke–Younger–Kasami Algorithm

$S \rightarrow V_a Y \mid SS \mid YS \mid V_a V_b$

$Y \rightarrow S V_b \mid Y V_b$

$V_a \rightarrow a$

$V_b \rightarrow b$

	1	2	3	4	5
a	V_a				
b	V_b				
b	V_b				
a	V_a				
b	V_b				

Cocke–Younger–Kasami Algorithm

$S \rightarrow V_a Y \mid SS \mid YS \mid V_a V_b$

$Y \rightarrow S V_b \mid Y V_b$

$V_a \rightarrow a$

$V_b \rightarrow b$

	1	2	3	4	5
a	V_a				
b	V_b				
b	V_b				
a	V_a				
b	V_b				

Cocke–Younger–Kasami Algorithm

$S \rightarrow V_a Y \mid SS \mid YS \mid V_a V_b$

$Y \rightarrow S V_b \mid Y V_b$

$V_a \rightarrow a$

$V_b \rightarrow b$

	1	2	3	4	5
a	V_a	S			
b	V_b				
b	V_b				
a	V_a				
b	V_b				

Cocke–Younger–Kasami Algorithm

$S \rightarrow V_a Y \mid SS \mid YS \mid V_a V_b$

$Y \rightarrow S V_b \mid Y V_b$

$V_a \rightarrow a$

$V_b \rightarrow b$

	1	2	3	4	5
a	V_a	S			
b	V_b				
b	V_b				
a	V_a	S			
b	V_b				

Cocke–Younger–Kasami Algorithm

$S \rightarrow V_a Y \mid SS \mid YS \mid V_a V_b$

$Y \rightarrow S V_b \mid Y V_b$

$V_a \rightarrow a$

$V_b \rightarrow b$

	1	2	3	4	5
a	V_a	S			
b	V_b	–			
b	V_b	–			
a	V_a	S			
b	V_b	–			

Cocke–Younger–Kasami Algorithm

$S \rightarrow V_a Y \mid SS \mid YS \mid V_a V_b$

$Y \rightarrow S V_b \mid Y V_b$

$V_a \rightarrow a$

$V_b \rightarrow b$

	1	2	3	4	5
a	V_a	S	Y		
b	V_b	–	–		
b	V_b	–	–		
a	V_a	S	–		
b	V_b	–	–		

Cocke–Younger–Kasami Algorithm

$S \rightarrow V_a Y \mid SS \mid YS \mid V_a V_b$

$Y \rightarrow S V_b \mid Y V_b$

$V_a \rightarrow a$

$V_b \rightarrow b$

	1	2	3	4	5
a	V_a	S	Y	–	S
b	V_b	–	–	–	
b	V_b	–	–	–	
a	V_a	S	–	–	
b	V_b	–	–	–	

Cocke–Younger–Kasami Algorithm

$$S \to V_a Y \mid SS \mid YS \mid V_a V_b$$

$$Y \to S V_b \mid Y V_b$$

$$V_a \to a$$

$$V_b \to b$$

	1	2	3	4	5
a	V_a	S	Y	–	S
b	V_b	–	–	–	
b	V_b	–	–	–	
a	V_a	S	–	–	
b	V_b	–	–	–	

$$O\left(|G| \times |w|^3 \right)$$

Machine Models

Consider the language $\left\{ w \in \{a, b\}^* : |w|_a = |w|_b \right\}$.

Machine Models

Consider the language $\{w \in \{a,b\}^* : |w|_a = |w|_b\}$.

```
def is_in_L(s: str) -> bool:
    counter = 0

    for ch in s:
        if ch == 'a':
            counter += 1
        elif ch == 'b':
            counter -= 1
        else:
            return False
    return counter == 0
```

Machine Models

Consider the language $\left\{w \in \{a, b\}^* : |w|_a = |w|_b\right\}$.

Counter γ

$$\delta(\gamma, a) = \gamma + 1$$

$$\delta(\gamma, b) = \gamma - 1$$

Machine Models

Consider the language $\left\{w \in \{a,b\}^* : |w|_a = |w|_b\right\}$.

Counter γ

$$\delta(\gamma, a) = \gamma + 1$$

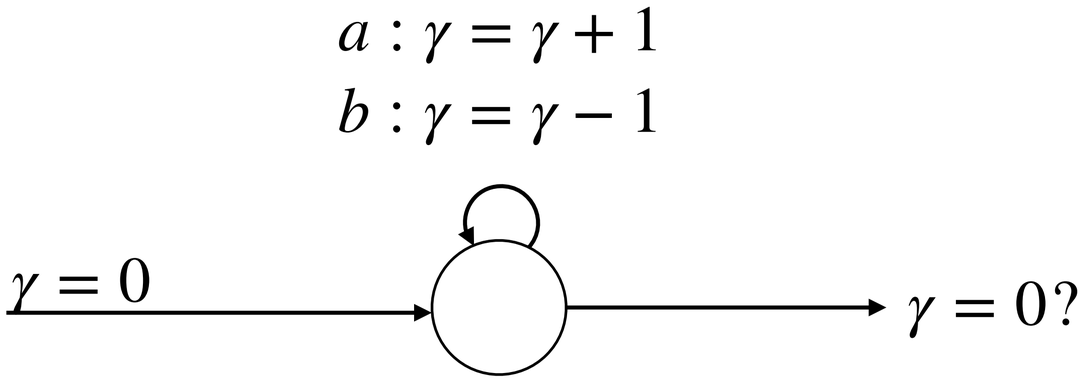
$$\delta(\gamma, b) = \gamma - 1$$

$s : \gamma = 0$

$F : \gamma = 0$

Machine Models

Consider the language $\left\{ w \in \{a, b\}^* : |w|_a = |w|_b \right\}$.

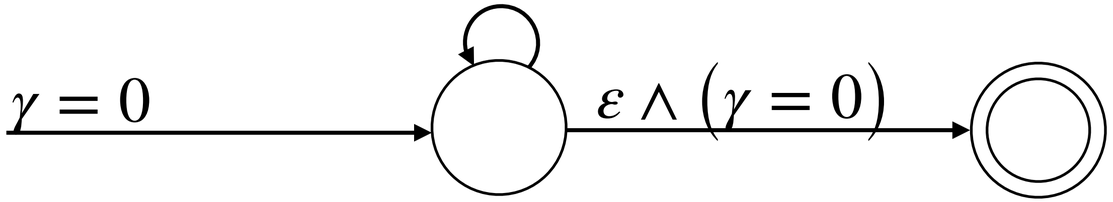


Machine Models

Consider the language $\left\{ w \in \{a, b\}^* : |w|_a = |w|_b \right\}$.

$$a \wedge (\gamma = _) : \gamma = \gamma + 1$$

$$b \wedge (\gamma = _) : \gamma = \gamma - 1$$

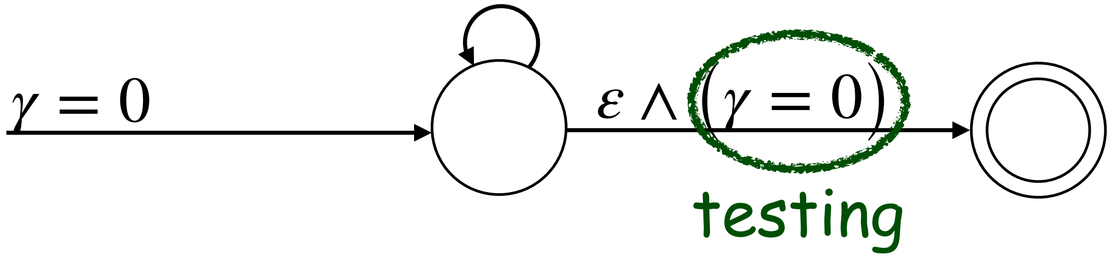


Machine Models

Consider the language $\{w \in \{a,b\}^* : |w|_a = |w|_b\}$.

$a \wedge (\gamma = _) : \gamma = \gamma + 1$ updates

$b \wedge (\gamma = _) : \gamma = \gamma - 1$

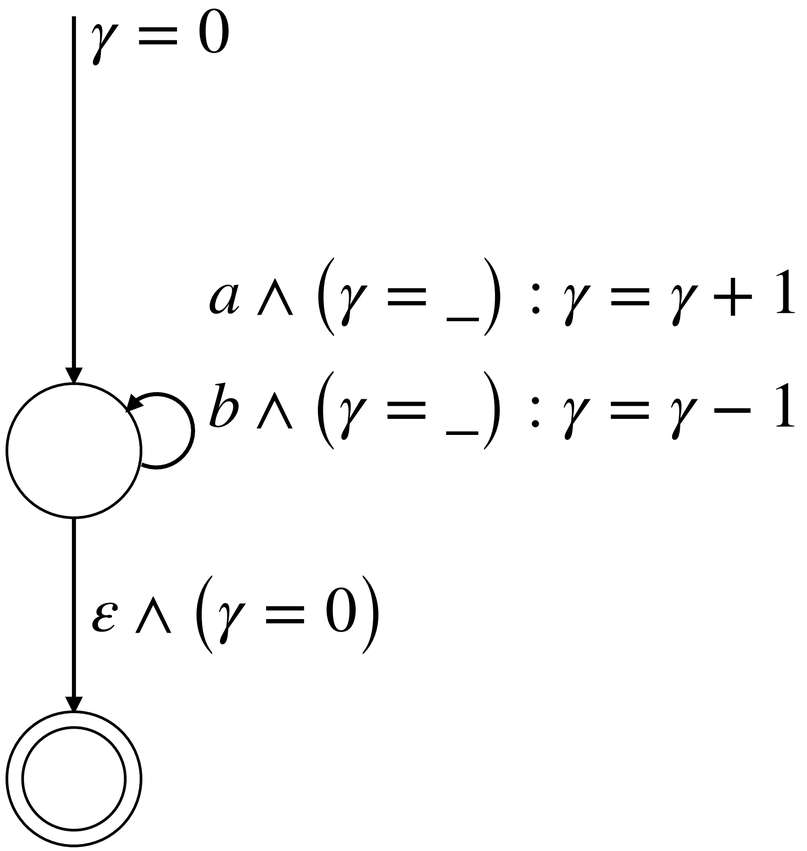


1-counter automata

A 1-counter automata is a 5-tuple:

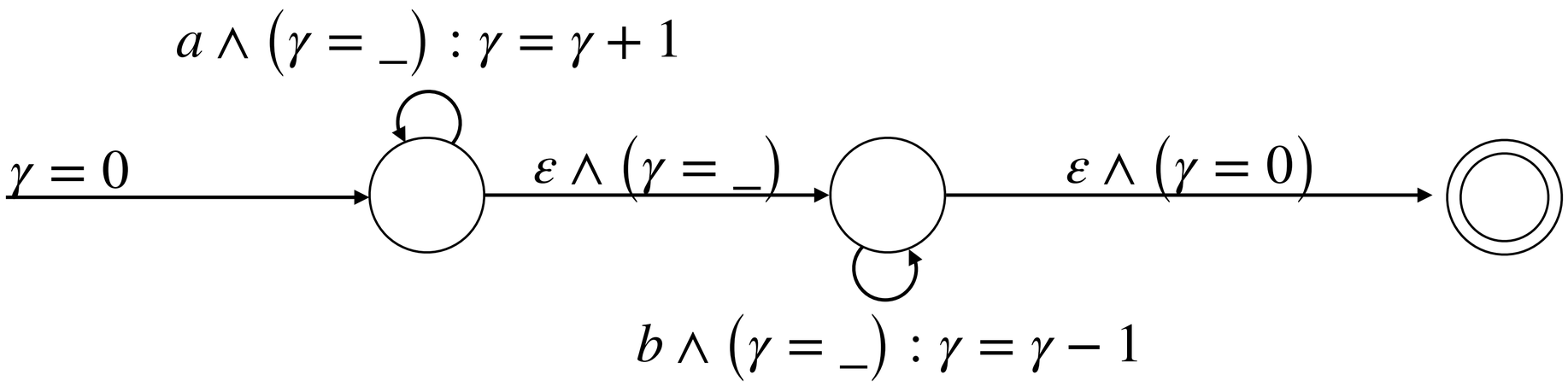
$$A = (Q, \Sigma, \delta, s, F), \text{ where:}$$

1. Q is a finite set of states
2. Σ is a finite alphabet
3. $\delta : Q \times \left(\Sigma \cup \{\varepsilon\}\right) \times \{0,1\} \to Q \times \{+1, -1, 0\}$ is a transition function
4. $s \in Q \times \mathbb{N}$ is an initial state, and
5. $F \subseteq Q$ is the set of final states.



1-counter automata

$$\{a^n b^n : \exists n \in \mathbb{N}\}$$



1-counter automata

Is 1-counter automata as expressive as a CFG?

1-counter automata

Is 1-counter automata as expressive as a CFG?

Consider $\{ww^R : w \in \Sigma^*\}$

1-counter automata

Is 1-counter automata as expressive as a CFG?

Consider $\{ww^R : w \in \Sigma^*\}$

Exercise:

92. For sake of contradiction, let there be a 1-counter automata with k states that accepts $\{ww^R : w \in \Sigma^*\}$. Show that after reading a word with n symbols, there at at most $O(kn)$ states. However there are at least 2^n words. Use it to derive a contradiction.

1-counter automata

Is 1-counter automata as expressive as a CFG?

Consider $\{ww^R : w \in \Sigma^*\}$

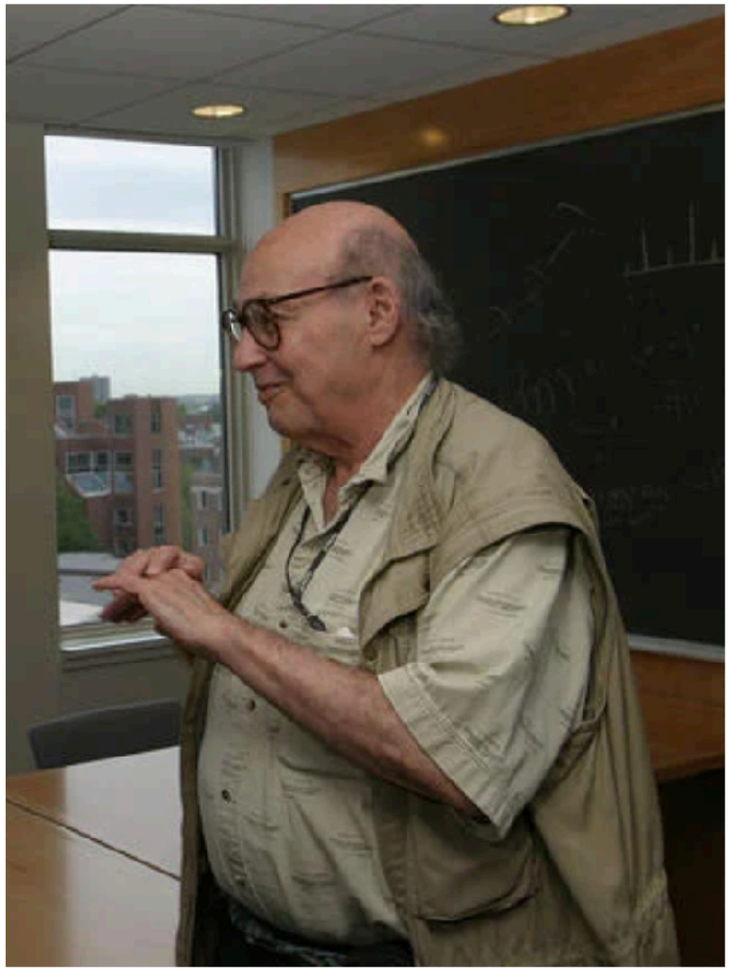
Exercises:

93. Give a 1-counter machine for recognising the set of well-balanced paratheses.

94. Define a two-counter machine. Show that a two counter machine can recognize the set $\{ww^R : w \in \Sigma^*\}$.

Minsky Machine

A Minsky machine (also called a counter machine or register machine) is a finite program operating on a finite number of unbounded natural-number registers.



Minsky Machine

A Minsky machine (also called a counter machine or register machine) is a finite program operating on a finite number of unbounded natural-number registers.

Jeffrey Epstein (left) talks with former Massachusetts Institute of Technology professor Marvin Minsky (right).

Machine Models

Consider $\{ww^R : w \in \Sigma^*\}$

We need to remember the count as well as what we saw.

1-stack automata

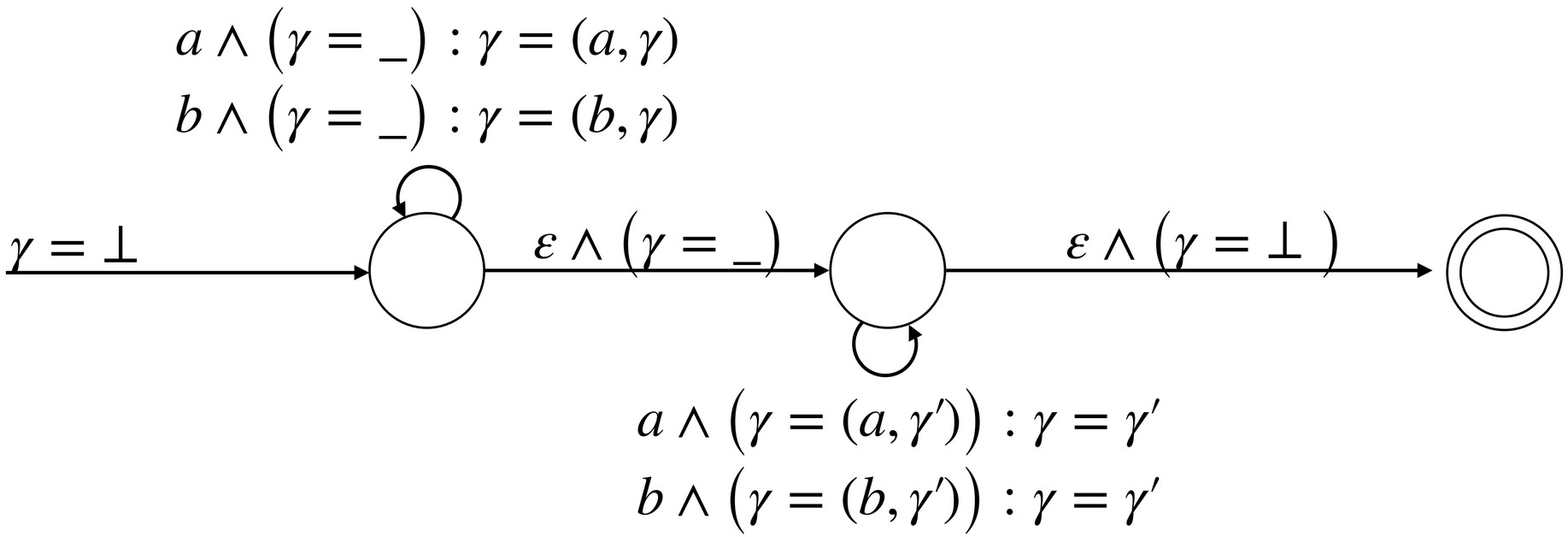
Consider $\{ww^R : w \in \Sigma^*\}$

We need to remember the count as well as what we saw.

1-stack automata

Consider $\{ww^R : w \in \Sigma^*\}$

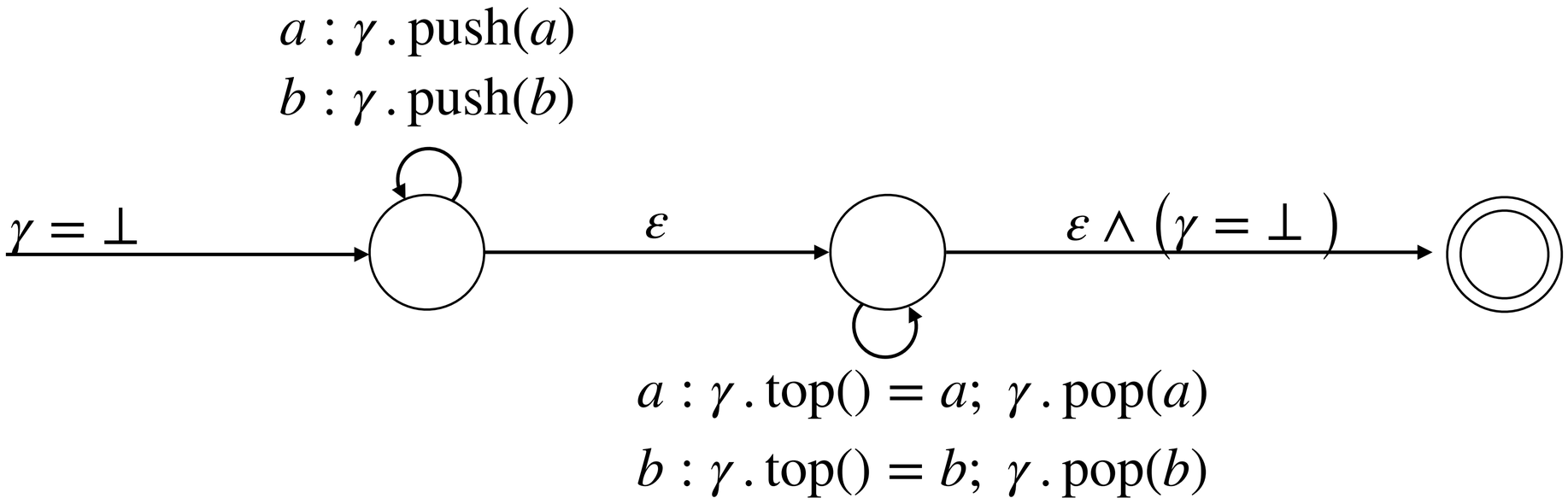
We need to remember the count as well as what we saw.



1-stack automata

Consider $\{ww^R : w \in \Sigma^*\}$

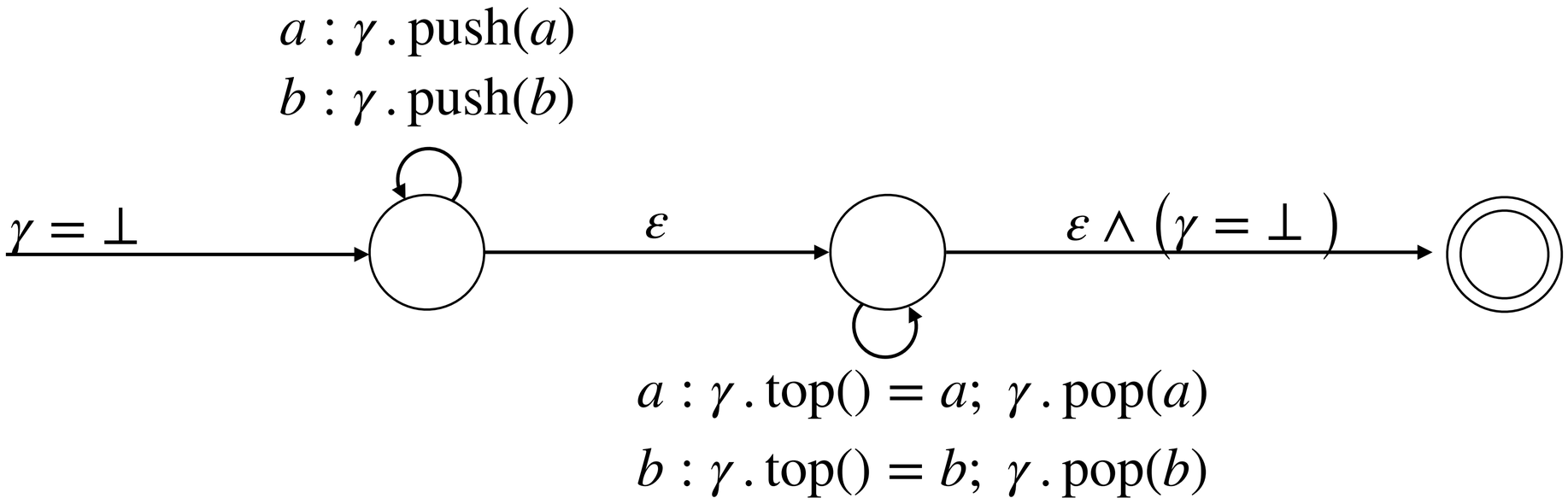
We need to remember the count as well as what we saw.



Push-down Automata (PDA)

Consider $\{ww^R : w \in \Sigma^*\}$

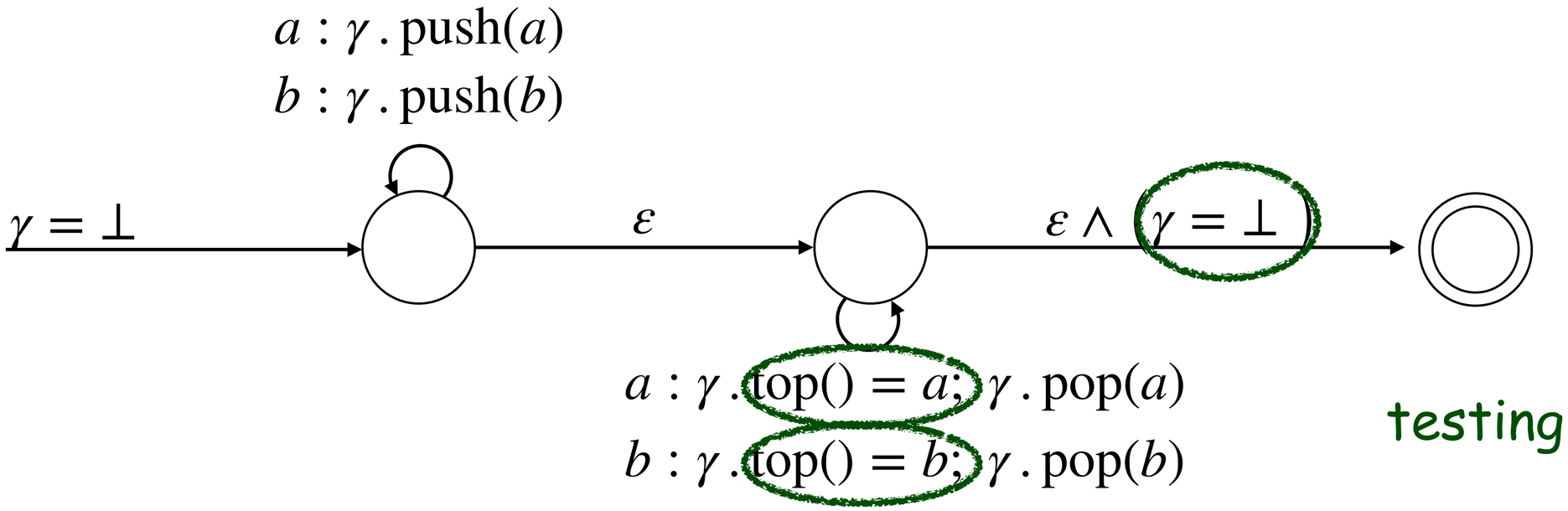
We need to remember the count as well as what we saw.



Push-down Automata (PDA)

Consider $\{ww^R : w \in \Sigma^*\}$

We need to remember the count as well as what we saw.

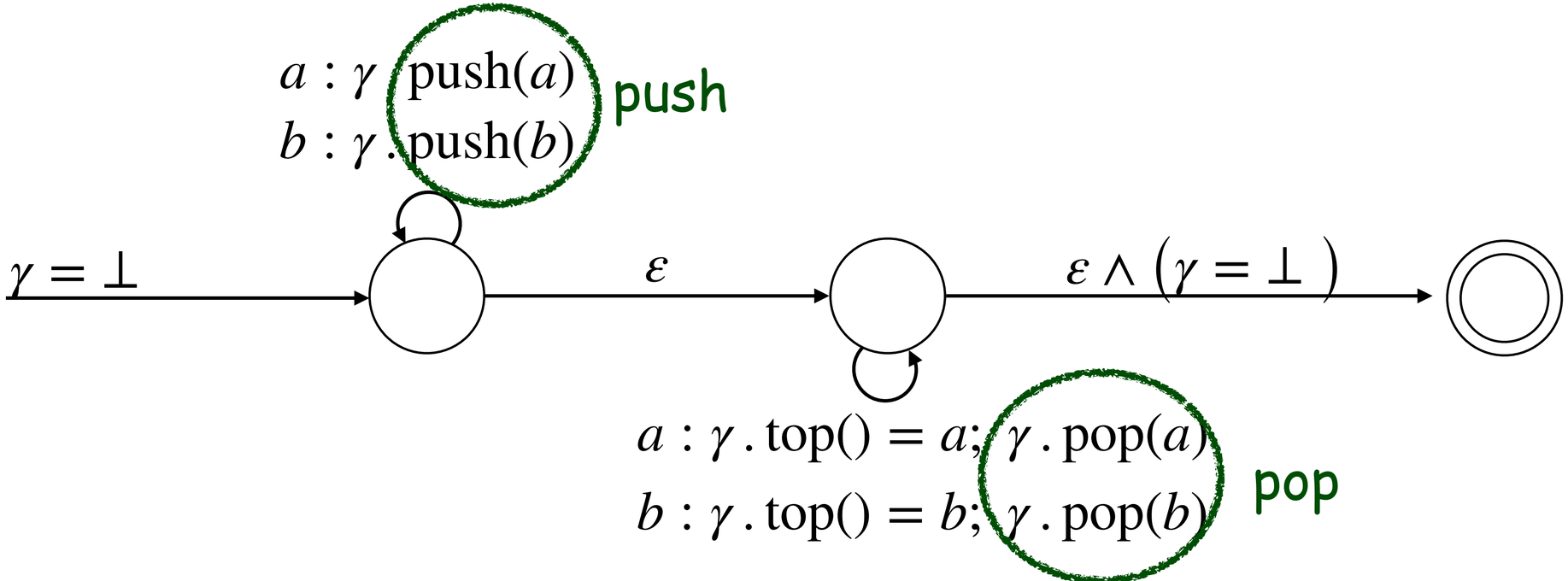


testing

Push-down Automata (PDA)

Consider $\{ww^R : w \in \Sigma^*\}$

We need to remember the count as well as what we saw.



Push-down Automata (PDA)

A (nondeterministic) pushdown automaton is a 7-tuple:

$$A = (Q, \Sigma, \Gamma, \delta, q_0, \perp, F)$$

where:

1. Q is a finite set of states.
2. Σ is a finite input alphabet.
3. Γ is a finite stack alphabet.
4. δ is the transition function.
5. $q_0 \in Q$ is the start state.
6. $\perp \in \Gamma$ is the initial (bottom) stack symbol.
7. $F \subseteq Q$ is the set of accepting states.

Push-down Automata (PDA)

A (nondeterministic) pushdown automaton is a 7-tuple:

$$A = (Q, \Sigma, \Gamma, \delta, q_0, \perp, F)$$

$$\delta : Q \times \left(\Sigma \cup \{\varepsilon\}\right) \times \Gamma \to \mathscr{P}\left(Q \times \Gamma^*\right)$$

If $(q', \alpha) \in \delta(q, a, X)$, then:

In state q, on reading a symbol $a \in \Sigma$ or ε, with $X \in \Gamma$on top of the stack, the machine may:

Move to state q' and replace X by $\alpha \in \Gamma^*$.

Push-down Automata (PDA)

Is push-down automata as expressive as a CFG?

Push-down Automata (PDA)

CFG $\Rightarrow$ PDA

Given a context free grammar $G = (N, \Sigma, P, S)$, consider:

$$M = (\{q_0, q, q_f\}, \Sigma, V \cup \Sigma, \delta, q_0, \perp, \{q_f\})$$

Push-down Automata (PDA)

CFG $\Rightarrow$ PDA

Given a context free grammar $G = (N, \Sigma, P, S)$, consider:

$$M = (\{q_0, q, q_f\}, \Sigma, V \cup \Sigma, \delta, q_0, \perp, \{q_f\})$$

Push start symbol:

$$\delta(q_0, \varepsilon, \perp) = \{(q, S\perp)\}$$

Push-down Automata (PDA)

CFG $\Rightarrow$ PDA

Given a context free grammar $G = (N, \Sigma, P, S)$, consider:

$$M = (\{q_0, q, q_f\}, \Sigma, V \cup \Sigma, \delta, q_0, \perp, \{q_f\})$$

For every production:

$A \rightarrow \alpha$, add

$$\delta(q, \varepsilon, A) \ni (q, \alpha)$$

This replaces A with α on the stack.

Push-down Automata (PDA)

CFG $\Rightarrow$ PDA

Given a context free grammar $G = (N, \Sigma, P, S)$, consider:

$$M = (\{q_0, q, q_f\}, \Sigma, V \cup \Sigma, \delta, q_0, \perp, \{q_f\})$$

For every terminal $a \in \Sigma$,

$$\delta(q, a, a) \ni (q, \varepsilon)$$

Consume the input and stack symbol together.

Push-down Automata (PDA)

CFG $\Rightarrow$ PDA

Given a context free grammar $G = (N, \Sigma, P, S)$, consider:

$$M = (\{q_0, q, q_f\}, \Sigma, V \cup \Sigma, \delta, q_0, \perp, \{q_f\})$$

When stack has only bottom symbol,
transition to final state:

$$\delta(q, \varepsilon, \perp) = \{(q_f, \perp)\}$$

Push-down Automata (PDA)

CFG $\Rightarrow$ PDA

High level idea: Look at the derivation.

$$S \Rightarrow AS \Rightarrow SS \Rightarrow aAbS \Rightarrow aAbbS \Rightarrow aSbbS \Rightarrow abbS \Rightarrow \cdots$$

Replicate it on the stack: When you see a non-terminal, expand it.

Push-down Automata (PDA)

CFG $\Rightarrow$ PDA

High level idea: Look at the derivation.

$$S \Rightarrow AS \Rightarrow SS \Rightarrow aAbS \Rightarrow aAbbS \Rightarrow aSbbS \Rightarrow abbS \Rightarrow \cdots$$

Replicate it on the stack: When you see a non-terminal, expand it.

When you see a terminal, consume the input and remove matching terminal.

Push-down Automata (PDA)

CFG $\Leftarrow$ PDA

A stack enforces nested structure:

Push something

Do some work

Eventually pop it

That is recursion.

Push-down Automata (PDA)

CFG $\Leftarrow$ PDA

Define grammar variables that mean:

A_{pq}: All strings that take the PDA from state p to state q while returning the stack to the same height.

Proof left out.

Push-down Automata (PDA)

$$\text{CFG} \iff \text{PDA}$$

Consequences:

1. Context-free languages are exactly those that require one stack of memory.
2. Context-free languages are exactly the languages whose structure can be described by single-level nested recursion.
3. There is no effective procedure to compute a minimal PDA.

Automata with Other Data Structures

1. Counters (registers)
2. Boolean flags
3. Finite Stacks
4. Multiple Stacks

1. Queues
2. Lists
3. Arrays
4. Heaps

1-Counter Automata

1-counter automata is same as PDA with one stack symbol.

$$\mathbf{Reg} \subsetneq \mathbf{1Counter} \subsetneq \mathbf{CFL}$$

2-Counter Automata

$\{a^n b^n c^n : n \in \mathbb{N}\}$ is not a context-free language.

Initialise counters to $(0,0)$.

While reading a, increment both counters.

On b decrement the first counter.

On c decrement the second counter.

$$\mathbf{Reg} \subsetneq \mathbf{1Counter} \subsetneq \mathbf{2Counter}$$

2-Counter Automata

$\{a^n b^n c^n : n \in \mathbb{N}\}$ is not a context-free language.

$$\mathbf{Reg} \subsetneq \mathbf{1Counter} \subsetneq \mathbf{kCounter}$$

Is $\mathbf{kCounter} \subseteq (\mathbf{k+1})\mathbf{Counter}$ a strict containment?

Boolean Flag Automata

Boolean Flag Automata

Suppose you have k flags. Denote each of the possible 2^k configurations of the flags with explicit states of the automaton.

Boolean Flag Automaton is a succinct DFA.

Finite Stack Automata

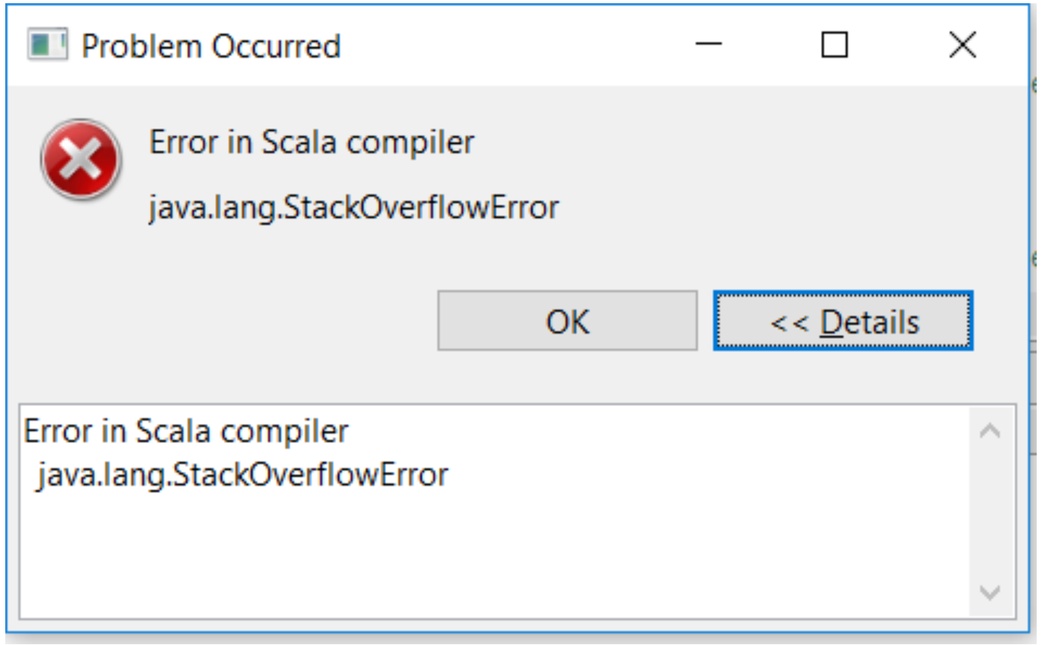
Finite Stack Automata

Suppose you have bound k flags. Denote each of the possible $\Gamma^{\leq k}$ configurations of the stack with explicit states of the automaton.

Finite Stack is a succinct DFA.

Finite Stack Automata

Suppose you have bound k flags. Denote each of the possible $\Gamma^{\leq k}$ configurations of the stack with explicit states of the automaton.



Multiple Stack Automata

Multiple Stack Automata

$$\mathbf{1Stack} \subsetneq \mathbf{2Stack} = \mathbf{kStack}$$

At any point, let the stacks be $(S_1, S_2, \cdots, S_k)$.

Multiple Stack Automata

$$\mathbf{1Stack} \subsetneq \mathbf{2Stack} = \mathbf{kStack}$$

At any point, let the stacks be $(S_1, S_2, \cdots, S_k)$.

In one stack store $S_1\#S_2\#\cdots\#S_k$. Keep the other empty.

Multiple Stack Automata

$$\mathbf{1Stack} \subsetneq \mathbf{2Stack} = \mathbf{kStack}$$

At any point, let the stacks be $(S_1, S_2, \cdots, S_k)$.

In one stack store $S_1\#S_2\#\cdots\#S_k$. Keep the other empty.

To update stack S_i, move all of
$S_1\#S_2\#\cdots\#S_{i-1}\#$ to the empty stack.

Update S_i, move all of $S_1\#S_2\#\cdots\#S_{i-1}\#$ back.

Queue Automata

2Stack $\subseteq$ Queue

At any point, let the stacks be (S_1, S_2).

In the queue, store $S_1 \# S_2 \#$

To pop or test S_1, dequeue/test the queue.

Queue Automata

2Stack $\subseteq$ Queue

At any point, let the stacks be (S_1, S_2).

In the queue, store $S_1\#S_2\#$

To pop or test S_1, dequeue/test the queue.

To push A on S_1, enqueue A. Dequeue and enqueue all of $S_1\#S_2\#$ sequentially.

Queue Automata

2Stack $\subseteq$ Queue

At any point, let the stacks be (S_1, S_2).

In the queue, store $S_1\#S_2\#$

To process S_2, dequeue all of $S_1\#$
(that is up to the first #).

Queue Automata

2Stack $\subseteq$ Queue

At any point, let the stacks be (S_1, S_2).

In the queue, store $S_1\#S_2\#$

To process S_2, dequeue all of $S_1\#$
(that is up to the first #).

At the end of each step, ensure the queue is of the form $S_1\#S_2\#$.

Queue Automata

2Stack $\supseteq$ Queue

Queue Automata

2Stack $\supseteq$ Queue

Start with all entries of the queue in one stack.

Other stack empty.

To dequeue, just pop. To test, just test the stack.

Queue Automata

2Stack $\supseteq$ Queue

Start with all entries of the queue in one stack.

Other stack empty.

To dequeue, just pop. To test, just test the stack.

To enqueue, transfer all elements to the empty stack.
Push, and then transfer elements back.

Exercises:

95. Prove that the following are equivalent:

    (a) **2Stack**

    (b) **kQueue**

    (c) Two stack and 3 counter Automata

    (d) List Automata

    (e) Heap Automata

96. Prove that any bounded data structure based state machine accepts only regular languages.